

NetVision

Plateforme BI de Supervision Réseau Télécom

Documentation Technique Complète

Projet	PFE 2025/2026 — Huawei Maroc
Client simulé	Opérateur télécom national (Maroc)
Stack	Python 3 · Flask · PostgreSQL · HTML/JS · Leaflet.js
Base de données	PostgreSQL 15 — Star Schema (Data Warehouse)
Équipements	120 équipements Huawei — BTS 2G/3G/4G · Routeurs · OLT · Antennes
Techniciens	30 techniciens répartis sur 10 secteurs du Maroc
Incidents	300 incidents générés — 8 types · 3 niveaux de sévérité
Date	04/06/2026

Table des Matières

1. Présentation Générale	3
2. Architecture Technique	4
3. Modèle de Données — Star Schema	5
4. Pipeline ETL et Génération de Données	6
5. API REST Flask — Endpoints	7
6. Module NOC Admin — Fonctionnalités	8
7. Module Technicien Terrain — Fonctionnalités	11
8. Logique Métier — Cycle de Vie d'un Ticket	13
9. Algorithme de Dispatch Intelligent	14
10. Gestion du Temps Réel et Automatisations	15
11. Étapes de Réalisation	16
12. Bilan Technique	17

1. Présentation Générale

NetVision est une plateforme de Business Intelligence dédiée à la supervision et au pilotage en temps réel des infrastructures réseau télécom Huawei au Maroc. Elle permet au personnel NOC (Network Operations Center) de détecter les incidents, d'assigner automatiquement les techniciens terrain et de suivre l'ensemble du cycle d'intervention depuis la détection jusqu'à la résolution.

Objectifs du projet

- Centraliser la supervision des 120 équipements Huawei répartis sur 10 secteurs du Maroc
- Automatiser le dispatch des 30 techniciens terrain selon leur compétence, proximité et disponibilité
- Fournir des KPI réseau en temps réel : MTTR, taux de résolution, incidents critiques
- Assurer un suivi complet du cycle de vie de chaque ticket d'intervention
- Permettre l'analyse historique via un planning Gantt journalier avec export Excel
- Détecter et traiter automatiquement les dépassements de durée d'intervention

Données simulées

Entité	Quantité	Détail
Secteurs	10	Casablanca, Rabat-Salé, Marrakech, Fès, Meknès, Agadir, Tanger, Oujda, Laâyoune, Kenitra,
Équipements	120	BTS 2G/3G/4G/5G, Routeurs NE40E-X8, Switches ATN910C, OLT Fibre, Antennes
Techniciens	30	Noms marocains, compétences RF/Fibre/IP, géolocalisés par secteur
Incidents	300	8 types, 3 sévérités, détectés dans les 7 derniers jours
Types incidents	8	PANNE_COMPLETE, COUPURE_FIBRE, PANNE_ALIM, DEGRADATION, SURCHARGE, IN

2. Architecture Technique

L'architecture suit un modèle en couches classique de Business Intelligence, depuis la génération des données jusqu'à la présentation dans les dashboards.

Couche	Technologie	Rôle
Génération données	Python (generate_data.py)	Simulation de 300 incidents, 120 équipements, 30 techniciens avec coordonnées
ETL	Python (etl_pipeline.py)	Extraction CSV, transformation, calcul durées estimées, chargement PostgreSQL
Data Warehouse	PostgreSQL 15	Star Schema : 5 tables dimensionnelles + 2 tables de faits
API Backend	Flask + psycpg2 + Flask-CORS	28 endpoints REST JSON, CORS activé, connexion directe PostgreSQL
Carte interactive	Leaflet.js + OpenStreetMap/CARTAGIS	Affichage équipements, incidents, techniciens géolocalisés
Dashboard NOC	HTML5 + Bootstrap Icons + JS	5 onglets : Dashboard, Carte, Incidents, Techniciens, Gantt
Interface Terrain	HTML5 + JS	Interface mobile-friendly pour techniciens terrain

Flux de données

- **generate_data.py** — Génère les CSV des entités (secteurs, équipements, techniciens, incidents)
- **etl_pipeline.py** — Charge les CSV dans PostgreSQL, calcule les durées estimées, crée le Star Schema
- **PostgreSQL** — Stocke toutes les données, exécute les requêtes KPI, maintient la cohérence
- **app.py (Flask)** — Expose les données via 28 endpoints REST, gère la logique métier (dispatch, tickets)
- **noc_admin.html** — Dashboard NOC : visualise KPI, carte, incidents, dispatch, Gantt
- **tech.html** — Interface technicien : ticket actif, mise à jour statut, historique

3. Modèle de Données — Star Schema

Le Data Warehouse est construit sur un schéma en étoile (Star Schema) avec une table de faits centrale (fact_incidents) entourée de tables dimensionnelles.

Table	Type	Colonnes principales	Description
dim_secteur	Dimension	id, nom, region, latitude, longitude	10 secteurs géographiques du Maroc
dim_equipement	Dimension	id, code, nom, type, statut, secteur_id, latitude, longitude, modele	120 équipements avec coordonnées GPS
dim_technicien	Dimension	id, matricule, nom, prenom, statut, secteur_id, latitude, longitude, competences	30 techniciens avec compétences et disponibilité
dim_temps	Dimension	id, date, jour, mois, annee, trimestre, jour_semaine	Dimensions temporelles pour analyses historiques
fact_incidents	Fait central	id, code_incident, type_incident, severite, secteur_id, latitude, longitude, date	300 incidents enregistrés
fact_tickets	Fait opérationnel	id, code_ticket, incident_id, technicien_id, date_arrivee_site	Tickets liés aux incidents
ticket_historique	Log	id, ticket_id, statut_avant, statut_apres, date_changement	3000 logs de statut de tickets

4. Pipeline ETL et Génération de Données

generate_data.py — Simulation des données

Ce script génère des données réalistes simulant un opérateur télécom marocain. Les coordonnées GPS sont réelles (villes marocaines), les noms de techniciens sont marocains, et les types d'incidents correspondent aux pannes réelles en télécommunications.

Type d'incident	Sévérité	Durée min (min)	Durée max (min)
PANNE_COMPLETE	CRITIQUE	120	480
COUPURE_FIBRE	CRITIQUE	180	600
PANNE_ALIM	CRITIQUE	240	720
DEGRADATION	MAJEUR	60	240
SURCHARGE	MAJEUR	45	180
ALARME_RESEAU	MINEUR	30	120
INTERFERENCE	MINEUR	20	90
MISE_A_JOUR	MINEUR	60	180

etl_pipeline.py — Chargement PostgreSQL

- Lecture des fichiers CSV depuis le dossier data/ (secteurs, équipements, techniciens, incidents)
- Validation et nettoyage des données (gestion colonnes manquantes, doublons)
- Calcul des durées estimées par type d'incident si absentes (plages réalistes)
- Création des tables PostgreSQL (DROP + CREATE avec contraintes FK)
- Chargement ordonné : dim_secteur → dim_equipement → dim_technicien → dim_temps → fact_incidents
- fact_tickets : table vide au départ — créée uniquement via le dispatch NOC
- Tous les incidents chargés avec statut OUVERT, tous les techniciens DISPONIBLES

5. API REST Flask — Endpoints

L'API Flask expose 28 endpoints REST organisés par domaine métier. Toutes les réponses sont en JSON avec dates en format ISO local (YYYY-MM-DDTHH:MM:SS) pour éviter les décalages UTC.

Endpoint	Méthode	Description
/api/health	GET	Vérification connexion PostgreSQL
/api/kpi	GET	KPI globaux : incidents ouverts/critiques/en cours, MTTR, techniciens disponibles
/api/incidents	GET	Liste incidents filtrés (sévérité, statut, secteur, type)
/api/incidents/critiques	GET	Incidents CRITIQUE encore actifs (OUVERT ou EN_COURS)
/api/incidents/par-secteur	GET	Agrégation incidents par secteur géographique
/api/incidents/par-reseau	GET	Agrégation par type de réseau (2G/3G/4G/Fibre/IP)
/api/incidents/par-heure	GET	Distribution des incidents par heure de la journée
/api/incidents/<id>	GET	Détail incident + ticket associé
/api/equipements	GET	Liste équipements filtrés (type, statut, secteur)
/api/equipements/stats	GET	Stats équipements par type (ACTIF/MAINTENANCE/HORS_SERVICE)
/api/techniciens	GET	Liste techniciens avec filtres
/api/techniciens/disponibles	GET	Techniciens disponibles (statut=DISPONIBLE, ticket_actif_id=NULL)
/api/techniciens/<id>	GET	Détail technicien + tickets récents + stats
/api/techniciens/<id>/ticket-actif	GET	Ticket actif non terminé d'un technicien
/api/carte	GET	Données carte : secteurs, équipements avec secteur_nom, incidents OUVERT+EN_COURS
/api/dispatch/<inc_id>	GET	Techniciens disponibles triés pour un incident donné
/api/dispatch/assigner	POST	Créer ticket + assigner technicien + notifier
/api/tickets	GET	Liste tickets avec filtres (statut, technicien, date)
/api/tickets/<id>	GET	Détail ticket + historique complet
/api/tickets/<id>/statut	PATCH	Mise à jour statut terrain (EN_ROUTE→SUR_SITE→EN_COURS→COMPLETE/INCOMPLET)
/api/tickets/incomplets	GET	Tickets clôturés INCOMPLET avec causes
/api/tickets/fermer-retards	POST	Fermeture automatique tickets dont durée estimée est dépassée
/api/gantt	GET	Planning journalier par date (?date=YYYY-MM-DD) — aujourd'hui ou historique
/api/performance	GET	Performance techniciens : tickets complétés, durée moyenne, note
/api/secteurs	GET	Liste des 10 secteurs
/api/secteurs/incidents	GET	Incidents agrégés par secteur avec MTTR

6. Module NOC Admin — Fonctionnalités

Le dashboard NOC Admin (noc_admin.html) est l'interface principale de supervision. Il est organisé en 5 onglets et se rafraîchit automatiquement toutes les 15 secondes.

Onglet 1 — Dashboard Global

- 6 KPI temps réel : incidents ouverts (décrémentés à l'assignation), incidents critiques OUVERT+EN_COURS (décrémentés à la résolution), en cours de maintenance, techniciens disponibles, MTTR moyen, durée estimée moyenne
- Graphique en anneau (donut) : répartition CRITIQUE/MAJEUR/MINEUR
- Graphique barres : incidents par heure de la journée (pic de charge)
- Graphique barres : incidents par type de réseau (2G, 3G, 4G, Fibre, IP, RF)
- Tableau des incidents critiques actifs avec bouton Dispatch direct
- Tableau de performance des techniciens (tickets complétés, note moyenne)
- Refresh automatique des KPI toutes les 15 secondes

Onglet 2 — Carte Géographique Leaflet

- Fond de carte sombre CARTO Dark Matter (OpenStreetMap)
- 120 équipements Huawei avec icônes par type (BTS, Routeur, OLT, Antenne) — couleur selon statut
- Incidents OUVERT et EN_COURS géolocalisés avec cercles pulsants (rouge=CRITIQUE, orange=MAJEUR, jaune=MINEUR)
- 30 techniciens avec statut coloré (vert=DISPONIBLE, orange=EN_INTERVENTION, gris=INDISPONIBLE)
- Panel latéral : détail équipement (type avec icône, modèle, secteur, coordonnées GPS)
- Panel dispatch : liste des techniciens disponibles triés par compétence + même secteur + note + distance
- Badge '★ Meilleur' sur le technicien recommandé, badge '✓ Compétent' si compétence correspond au type de panne
- Bouton 'Dispatcher' → crée le ticket, assigne le technicien, met à jour les statuts en base
- Après dispatch : flyTo sur l'équipement exact de l'incident (pas les coordonnées génériques)

Onglet 3 — Gestion des Incidents

- Liste filtrée : sévérité (CRITIQUE/MAJEUR/MINEUR), statut (OUVERT/EN_COURS), secteur, type d'incident
- Tableau avec badges colorés par sévérité
- Clic sur incident → flyTo sur la carte + ouverture panel dispatch
- Compteurs dynamiques : nb Critiques / Majeurs mis à jour en temps réel

Onglet 4 — Techniciens

- Liste complète des 30 techniciens avec statut, matricule, secteur
- Filtrage par statut (DISPONIBLE/EN_INTERVENTION/INDISPONIBLE) et secteur
- Clic → flyTo sur la carte + panel détail technicien
- Stats : total interventions, complétées, incomplètes, durée moyenne, note

Onglet 5 — Gantt Planning Journalier

- Planning journalier par technicien sur axe horaire 8h-17h avec pause déjeuner 13h-14h
- Navigation par date (← →) : aujourd'hui = actifs, dates passées = historique COMPLETE/INCOMPLET
- Barres colorées par statut : mauve (ASSIGNE/en attente), bleu (EN_ROUTE), orange (SUR_SITE), cyan (EN_COURS), vert (COMPLETE), rouge (INCOMPLET)
- Barre rouge clignotante si durée estimée dépassée
- Bouton ☎ Appeler visible pour chaque technicien (lien tel: pour mobile/Skype/Teams)
- Badge retard + bouton appel dans la colonne si dépassement détecté
- Tooltip au survol : incident, type, sévérité, heures assignation/arrivée/début/fin, durée réelle calculée, durée estimée
- Durée réelle = date_fin_intervention - date_debut_intervention (ou date_arrivee_site si pas de début)
- Auto-refresh toutes les 10 secondes avec popup notification si changement de statut terrain détecté
- Popup de changement statut : nom technicien, nouveau statut avec icône et couleur, bouton Appeler
- Export Excel (CSV UTF-8 BOM) : toutes les colonnes (ID ticket, technicien, dates, durées réelles/estimées, cause)
- Fermeture automatique des tickets en retard toutes les 30 secondes (POST /api/tickets/fermer-retards)

7. Module Technicien Terrain — Fonctionnalités

L'interface terrain (tech.html) est optimisée pour une utilisation sur mobile ou tablette. Elle se rafraîchit toutes les 10 secondes pour détecter les nouveaux tickets assignés.

Identification

- Sélecteur de profil technicien au démarrage (liste chargée depuis /api/techniciens)
- Affichage nom, matricule, secteur dans le header
- Bouton 'Changer de technicien' disponible en permanence

Popup de notification nouveau ticket

- Détection automatique toutes les 10s via `_prevTicketId` (undefined → null → id)
- Popup plein écran avec : code ticket, type incident, sévérité (couleur adaptée), secteur, durée estimée, date d'assignation
- Fermeture par X ou clic sur l'overlay
- Toast de notification complémentaire

Ticket actif — Cycle terrain

Étape	Statut terrain	Action technicien	Données enregistrées en DB
0 — Assignation	NULL / ASSIGNE	Reçoit popup notification	date_assignment, duree_estimee_min
1 — Départ	EN_ROUTE	'Je pars maintenant'	Barre mauve dans le Gantt
1 bis	SUR_SITE	'Déjà sur site' (direct)	date_arrivee_site = NOW()
2 — Arrivée	SUR_SITE	'Je suis arrivé sur site'	date_arrivee_site = NOW()
3 — Intervention	EN_COURS	'Je commence l'intervention'	date_debut_intervention = NOW()
4a — Succès	COMPLETE	'Terminé avec succès'	date_fin_intervention, duree_intervention_min calculée
4b — Échec	INCOMPLET	'Incomplet' + cause obligatoire	cause_incomplete, duree_intervention_min calculée

Contrôles et blocages

- Timeline visuelle : points EN_ROUTE → SUR_SITE → INTERVENTION → TERMINÉ avec couleurs progressives
- Boutons séquentiels : seulement l'étape suivante est proposée
- Blocage complet si `date_assignment + duree_estimee_min < maintenant` (durée dépassée)
- Message 'Actions bloquées — Le NOC doit clôturer' si retard
- Toast d'avertissement 10 minutes avant la fin estimée
- Toast d'erreur si durée dépassée

Historique

- 10 dernières interventions COMPLETE et INCOMPLET
- Date d'assignation, durée réelle, note, cause si INCOMPLET

8. Logique Métier — Cycle de Vie d'un Ticket

Chaque ticket suit un cycle de vie strict depuis sa création jusqu'à sa clôture. Toutes les transitions sont tracées dans ticket_historique avec horodatage et auteur.

Transition	Déclencheur	Actions en base de données
Création (ASSIGNE)	Admin NOC clique 'Dispatche'	INSERT fact_tickets (statut_terrain=NULL), UPDATE dim_technicien (statut=EN_INTE
NULL → EN_ROUTE	Technicien : 'Je pars maintenant'	UPDATE fact_tickets (statut_terrain=EN_ROUTE), INSERT ticket_historique
* → SUR_SITE	Technicien : 'Arrivé sur site'	UPDATE fact_tickets (statut_terrain=SUR_SITE, date_arrivee_site=NOW()), INSERT t
SUR_SITE → EN_COURS	Technicien : 'Je commence'	UPDATE fact_tickets (statut_terrain=EN_COURS, date_debut_intervention=NOW()), IN
EN_COURS → COMPLETE	Technicien : 'Terminé'	UPDATE fact_tickets (date_fin_intervention=NOW(), duree_intervention_min=calculée)
* → INCOMPLET	Technicien + cause obligatoire	UPDATE fact_tickets (INCOMPLET, cause_incomplete, duree_intervention_min), UPD
Auto → INCOMPLET	Système (dépassement durée)	POST /api/tickets/fermer-retards : UPDATE fact_tickets (INCOMPLET, 'Clôture automa

9. Algorithme de Dispatch Intelligent

Le dispatch sélectionne le meilleur technicien disponible pour chaque incident en combinant 4 critères ordonnés par priorité.

Priorité	Critère	Détail
1 (le plus important)	Compétence correspondante	Mapping type incident → compétence requise : COUPURE_FIBRE → Fibre, INTERFER
2	Même secteur géographique	Technicien dans le même secteur que l'équipement en panne
3	Meilleure note de performance	note_perf (1-5) calculée sur l'historique des interventions
4 (le moins important)	Distance la plus courte	Distance euclidienne approchée : $111 * \sqrt{(\text{lat1}-\text{lat2})^2 + (\text{lng1}-\text{lng2})^2}$ km

Résultat du dispatch

- Badge '★ Meilleur' affiché sur le premier technicien de la liste (recommandé)
- Badge '✓ Compétent' sur tous les techniciens ayant la compétence requise
- Section 'Même secteur' séparée de 'Autres secteurs' pour la lisibilité
- Techniciens EN_INTERVENTION (avec ticket actif) filtrés et non proposés
- La compétence requise est affichée en haut du panel de dispatch

10. Gestion du Temps Réel et Automatisations

Mécanisme	Fréquence	Déclencheur	Action
Refresh KPI	15 secondes	setInterval JS	GET /api/kpi → mise à jour k-crit, k-ouv, k-encours, k-dispo
Refresh Gantt	10 secondes	startGanttAutoRefresh()	GET /api/gantt?date= → re-render barres + détection changements
Popup changement statut	10 secondes	Comparaison _prevState	Si statut terrain a changé → popup avec nom tech, statut, bouton ap
Fermeture auto retards	30 secondes	setInterval global	POST /api/tickets/fermer-retards → clôture INCOMPLET si date_ass
Popup nouveau ticket	10 secondes	Comparaison _prevTicket	Si nouveau ticket détecté pour le technicien → popup plein écran
Toast avertissement fin	10 secondes	checkRetard()	Si date_assignment + duree_estimee - 10min < NOW() → toast oran
Toast dépassement	10 secondes	checkRetard()	Si date_assignment + duree_estimee < NOW() → toast rouge + bloc
Arrêt auto-refresh Gantt	À la navigation	ganttPrevDay()/Next()	Si date historique → stopGanttAutoRefresh(), si retour aujourd'hui →

11. Étapes de Réalisation

■ Phase 1 — Analyse et conception

- Analyse des besoins métier d'un opérateur télécom marocain
- Définition du périmètre : 10 secteurs, 8 types d'incidents, 3 sévérités
- Conception du Star Schema PostgreSQL (5 dimensions + 2 faits)
- Définition des KPI cibles : MTTR, taux résolution, performance techniciens
- Choix de la stack : Python/Flask/PostgreSQL/Leaflet.js

■ Phase 2 — Génération et ETL

- Développement de generate_data.py : 300 incidents, 120 équipements, 30 techniciens avec GPS réels du Maroc
- Développement d'etl_pipeline.py : chargement CSV → PostgreSQL avec validation
- Calcul automatique des durées estimées par type d'incident
- Création du Star Schema avec contraintes FK et index
- Génération des données de dim_temps sur 3 ans

■ Phase 3 — API REST Flask

- Développement de 28 endpoints REST couvrant tous les domaines métier
- Serialisation des dates en ISO local (évite décalage UTC+1)
- Gestion CORS pour accès cross-origin depuis les interfaces HTML
- Algorithme de dispatch : tri par compétence + secteur + note + distance
- Endpoints spécialisés : /api/gantt?date=, /api/tickets/fermer-retards, /api/techniciens//ticket-actif
- Calcul durée réelle d'intervention lors de la clôture COMPLETE/INCOMPLET

■ Phase 4 — Dashboard NOC Admin

- Interface 5 onglets : Dashboard, Carte, Incidents, Techniciens, Gantt
- Intégration Leaflet.js avec fond CARTO Dark Matter
- Graphiques KPI : donut sévérités, barres par heure et par réseau
- Système de dispatch avec panel glissant et badges compétence
- Gantt planning avec barres colorées par statut, tooltip, export Excel
- Auto-refresh 10s/15s/30s avec popups de notification
- Fermeture automatique des tickets en retard côté frontend

■ Phase 5 — Interface Technicien Terrain

- Interface responsive optimisée mobile
- Cycle terrain séquentiel : boutons adaptés à l'étape courante
- Popup plein écran à la réception d'un nouveau ticket
- Blocage des actions si durée estimée dépassée
- Historique des interventions avec timeline modale
- Avertissement 10 min avant fin estimée

■ Phase 6 — Corrections et optimisations

- Résolution du problème de timezone UTC+1 : extraction ISO directe sans new Date()
- Fix du calcul de durée réelle (parse_dt() pour éviter erreurs str→datetime)
- Correction du cycle de vie : statut_terrain NULL au dispatch (pas EN_ROUTE automatique)
- Fix des KPI : incidents_critiques = OUVERT+EN_COURS seulement (décrément à COMPLETE)
- Historique Gantt : /api/gantt?date= avec filtre COMPLETE/INCOMPLET pour dates passées
- Export Excel avec dates textuelles (apostrophe anti-##### Excel)

12. Bilan Technique

Critère	Résultat	Détail
Couverture fonctionnelle	Complète	Tous les modules métier implémentés : supervision, dispatch, terrain, Gantt, historique
Endpoints API	28 endpoints	Tous les domaines couverts, pagination, filtres multiples
Temps réel	Oui (10-30s)	Auto-refresh KPI, Gantt, détection changements, fermeture auto retards
Timezone	Corrigé	Dates ISO locales, zéro toLocaleString(), LOCALTIMESTAMP en PostgreSQL
Durée réelle	Calculée	date_fin - date_debut_intervention, stockée en DB, affichée dans Gantt et export
Dispatch intelligent	4 critères	Compétence + Secteur + Note de performance + Distance GPS
Notifications	Temps réel	Popups changement statut, nouveau ticket, retard, avertissement fin imminente
Historique	Par date	Gantt consultable pour n'importe quelle date passée
Export	Excel/CSV	CSV UTF-8 BOM, dates textuelles, durées réelles calculées
Appels téléphoniques	Via tel:	Lien tel: → mobile direct, PC via Skype/Teams
Sécurité	Basique	Sans authentification (démonstration PFE) — CORS configuré